一级主题	二级模块	核心概念	主要作用与联系
Go 并发编程	并发执行单元	Goroutine	Go 的轻量级并发执行单元，由 Go Runtime 管理，通过 M-to-N 映射到操作系统线程
Go 并发编程	共享内存同步（Shared Memory）	Mutex	互斥锁，保护共享资源，避免多个 Goroutine 同时进入临界区
		RWMutex	读写锁，允许多个读操作并发，写操作具有独占性
		Cond	条件变量，通过 Wait、Signal、Broadcast 协调 Goroutine 执行
		Atomic	原子操作，保证相关读写操作不可被中断
		Once	保证某个函数只执行一次，常用于共享变量初始化
		WaitGroup	等待多个 Goroutine 完成任务，通过 Add、Done、Wait 实现任务同步
Go 并发编程	消息传递同步（Message Passing）	Channel	在不同 Goroutine 之间传递数据和状态
		Unbuffered Channel	无缓冲通道，发送与接收需要同步配合
		Buffered Channel	带缓冲通道，利用缓冲区暂存数据
		Select	同时等待多个 Channel 操作，多个 case 可执行时随机选择
		Context	在多个 Goroutine 之间传递请求相关的数据或元数据
		Pipe	在 Reader 和 Writer 之间传输数据流
并发潜在问题	共享内存相关	Data Race	多个 Goroutine 并发访问共享变量，可能产生数据竞争
	消息传递相关	Blocking	Channel 阻塞、nil Channel 等可能导致 Goroutine 无法继续执行
	消息传递相关	Runtime Panic	向已关闭的 Channel 发送数据或重复关闭 Channel
	并发控制相关	非确定性	Select 在多个 case 同时满足条件时随机选择，可能导致非预期行为
实际应用研究	研究对象	Docker、Kubernetes	容器系统
		etcd	键值存储系统
		CockroachDB、BoltDB	数据库系统
		gRPC-Go	RPC 通信库
使用模式分析	静态分析（Static Analysis）	Goroutine Creation Sites	统计源代码中创建 Goroutine 的位置及创建方式
	动态分析（Dynamic Analysis）	Runtime Goroutines	统计程序运行时 Goroutine 的数量和执行时间
	并发原语分析	Primitive Usage	统计 Shared Memory 和 Message Passing 原语的使用数量及比例
研究结论	Observation 1	Goroutine 使用特点	Goroutine 生命周期较短，但创建更加频繁
	Observation 2	并发原语使用特点	Shared Memory 仍被大量使用，Message Passing 也十分普遍
	Implication 1	潜在影响	大量 Goroutine 和多种并发原语可能增加并发 Bug 风险